Meeting Script — what to show, say, and run

Bind Pratap Singh & Suhani Deepak Agrawat · Advisor: Dr. Sweta Mishra · September 2026

> **Open with (10 sec):** "Last time we'd finished the literature review and were about to start the prototype. Since then we've built and tested the **whole pipeline**, generated a **dataset**, and trained a model chosen by evidence — and it all runs. Let me walk you through it."

1 The architecture — show the flowchart, narrate 7 stages

SHOW The pipeline diagram — GitHub README (top), or Figure 1 of the Approach PDF.

SAY "The extension version is the **only** thing that differs between the two runs — so any difference in behaviour is the extension, not web noise."

TRACE

1. **Input** — two versions of one extension: v1 trusted, v2 the update.
2. **Unpack + instrument** — unzip, read the manifest, inject monitoring into the extension's own code (content scripts *and* the MV3 service worker).
3. **Sandbox capture ×2** — headless Chromium; a scripted user logs into a *fake* bank + webmail; network egress blocked; records four channels: **network, DOM, cookies/storage, chrome.* API**.
4. **Two behavioural records** — T1 and T2: exactly what each version did.
5. **The diff** — Δ = f(v2) − f(v1) → 24 numbers (new hosts, cookie reads, read-then-upload flows, injected nodes, permission escalation).
6. **Three scorers** — transparent rules · ML model · anomaly detector.
7. **Verdict + evidence** — BENIGN / SUSPICIOUS / MALICIOUS, cross-checked against a collector log that says whether data really left the browser.

2 The dataset — show it, then give the numbers

SHOW Open `data/dataset/deltas.csv` in Excel / VS Code. "Each row is one update — v1→v2 of an extension — reduced to 27 numbers plus its label."

POINT A **malicious** row → `cookie_bulk_reads=1`, `exfil_flows_third_party=1`, a new `cookies` permission. A **benign** row → mostly zeros.

RUN

```
python -c "import pandas as pd; d=pd.read_csv('data/dataset/deltas.csv'); print(d.shape);
print(d.label.value_counts()); print(d.source.value_counts())"
```

The numbers to quote: **~1,000** synthetic update pairs · **~250** distinct extensions · **27** features each · 12 personas · **9** attack families · ~half obfuscated + stealth variants, built with deliberate overlap so it's **hard**. **Real data pulled today: 7,382** confirmed-malicious extension IDs (public IOC database) + real extensions via our collector (Dark Reader, Grammarly, Stylus).

SAY **Synthetic (~1,000):** "A generator with a dozen realistic extension personas. For each we make a v1, then a benign update or a malicious one by injecting one of nine attack families. The label lives in the *difference*, not the raw counts, and we deliberately add overlap — benign updates that look suspicious, stealth attacks that hide the obvious tells — so the ML task is hard, not a trivial threshold."

SAY **Real data:** "We pulled 7,382 confirmed-malicious extension IDs from a public IOC database, and downloaded real extensions (Dark Reader, Grammarly, Stylus) through our own collector — proving we can fetch and analyse real-world extensions, not just generated ones."

> **Be upfront (do not overclaim):** we do **not** yet have real *store* extensions — the "real" element today is that the pipeline runs in a real browser. **SAY:** "Collecting real Web-Store extension pairs is the immediate next step — the collector is already built (GitHub releases, archives, and weekly snapshots of installed extensions)."

3 The model — show it's chosen by evidence (handles the "XGBoost?" question)

RUN `python -m engine.ml.bakeoff` → prints the five-model table and "Does ML earn its place? YES".

SAY "We race five models. On the larger, harder dataset **XGBoost — the tabular state-of-the-art you asked us to add — wins** (PR-AUC 0.97). This is where ML earns its place: to catch 90% of attacks the rules alone flag **52%** of benign updates; the model flags **~7%**. The genuine further step — a sequence/graph model — is Project-2."

4 Live demo — run at least the first

RUN `python scripts/demo_offline.py` instant, no browser → BENIGN / SUSPICIOUS / MALICIOUS, all correct (safest)

RUN `python scripts/demo.py` finale, if the room allows → a payload steals cookies live, we catch it, collector confirms the leak

SAY "This is not a plan — it runs. Here's the dataset, here's the model chosen by evidence, and here it is catching a live attack a static scanner would miss."

5 Close — the honest caveat (reads as rigour) + what's next

SAY "These numbers are from a synthetic corpus plus live captures, so they're cleaner than the real world. What holds regardless is the **relationship**: dynamic beats static on obfuscation, the model beats the rules on false positives, the anomaly layer catches families it never saw. Next: real update pairs, then the Firefox second engine as Project-2."

---

extdrift · CSD493 Project-1 · Meeting script, September 2026. Figures reflect the current synthetic corpus + live browser captures. No real malware runs outside the sealed, network-restricted sandbox.